

mai. 4, 2026

Padrões para Domínios Ricos - Parte 1

convidado Bruno Joaquim Cassia Dobler Fabio Diniz Francisco Schneider
Gabriel Severo Gabriel kohlrausch Ruan Rodrigues
aline.bianchini@t-systems.com ana.santos@t-systems.com
analista.kuehn@outlook.com anderson.hilario@t-systems.com
anderson.perim@t-systems.com andersongabriel_maria@hotmail.com
anselmogabriel421@gmail.com bmf.pedro@gmail.com
bruna.nery@t-systems.com bruno.destro@t-systems.com
caique.rodrigues@t-systems.com d.silva@t-systems.com
danilosilva2017@hotmail.com desenvolvedor@tidsoftware.com.br
douglas.nishiyama@t-systems.com dpsnbsb79@gmail.com
eduardo.kubota@t-systems.com eudy.amparo@t-systems.com
fabio.marinelli@t-systems.com farley.t.i@hotmail.com
felipeneves089@gmail.com gustavo.ferrazfontes@gmail.com
hewerton.souza@t-systems.com i.parmigiani@t-systems.com
igor.romulo.santos@gmail.com jeanderson.silva@t-systems.com
jl.oliveir@gmail.com jonathan.jimenez@iatec.com karinetrevisann@gmail.com
l.regioli@outlook.com leandro.piqueira@t-systems.com
lucas.p.oliveira@outlook.pt lxsilvareis@gmail.com
marcelohcordeiro@gmail.com marcio@almob.com.br
marcos.ammon@gmail.com moraesguga@gmail.com n.segeti@t-systems.com
nadson.dr11@gmail.com Paulo Scatena rafael.dias@t-systems.com
regis.maioli@t-systems.com reinaldo.rodrigues@t-systems.com
renan-silva@t-systems.com renan.baptista.cabral@gmail.com
rodrigo_7sch@hotmail.com tatidornel@gmail.com
thiago.fernandes@t-systems.com victor.prospero@t-systems.com
victor.santos@t-systems.com victor.souza@t-systems.com
vinicius.bonicio@t-systems.com webdeveloperrm@gmail.com
wesley.boccaletto@t-systems.com wsilva@t-systems.com
fabiano.felgas@gmail.com

Anexos  Padrões para Domínios Ricos - Parte 1

Resumo

O encontro detalhou padrões de projeto criacionais e estruturais para aprimorar a arquitetura e modularidade via encapsulamento.

Padrões de Criação

Padrões como Factory e Builder simplificam a criação de objetos e promovem o desacoplamento. O Singleton deve ser implementado via injeção de dependência para garantir segurança em concorrência.

Padrões Estruturais

O Decorator permite adicionar funcionalidades através de composição de objetos. O Facade atua como um orquestrador que simplifica interfaces para subsistemas complexos e facilita a refatoração de sistemas legados.

Próximas etapas

- ☐ [Francisco Schneider] Discutir Chain: Falar detalhadamente sobre Chain of Responsibility. Abordar as diferenças em relação ao Decorator na quarta-feira.
- ☐ [Francisco Schneider] Compartilhar Repositório: Enviar o link do repositório contendo o código de exemplo do Decorator.

Detalhes

- **Estrutura da Aula e Tipos de Padrões de Projeto:** Francisco Schneider iniciou a discussão sobre Padrões de Projeto, esclarecendo que a conversa seria dividida em duas sessões, sendo a próxima na quarta-feira. O objetivo não é ser exaustivo, mas focar nos padrões mais utilizados. Serão abordados os

três tipos de padrões: Criacionais e Estruturais (vistos na sessão atual), e Comportamentais (vistos na quarta-feira) ([00:00:00](#)).

- **Conceito de Padrões de Projeto:** Padrões de projeto são descritos como soluções típicas ou "receitas de bolo" para resolver problemas comuns no design de software, e não na arquitetura ([00:06:08](#)). Embora as implementações possam variar, o cerne e o problema que o padrão se propõe a resolver devem ser preservados. Alguns padrões que não estão na lista original foram incorporados pela comunidade, e muitos deles já são utilizados diariamente, muitas vezes sem o conhecimento dos desenvolvedores ([00:07:37](#)).
- **Padrões Criacionais e Desacoplamento:** Padrões criacionais auxiliam na criação de objetos, muitas vezes permitindo a criação sem o uso explícito da palavra-chave `new`. O benefício principal desses padrões é garantir um maior desacoplamento entre os componentes ([00:09:10](#)).
- **Simple Factory (Fábrica Simples):** O Simple Factory foi apresentado como um tipo de padrão, apesar de não ser um padrão oficial, devido à sua relevância e uso cotidiano ([00:10:43](#)). Este padrão facilita a criação de objetos, geralmente exposto através de métodos estáticos ([00:12:21](#)). Apesar de não ser formal, ele resolve a maior parte dos problemas e serve como um primeiro degrau na complexidade dos padrões de criação, tornando a criação de objetos mais representativa e legível para o cliente ([00:13:54](#)).
- **Discussão sobre Múltiplos Construtores e Modelagem de Domínio:** Francisco Schneider observou que múltiplos construtores podem ser um indício de má modelagem de domínio ou que a classe está excessivamente grande, com responsabilidades a mais do que deveria ([00:16:50](#)) ([00:19:45](#)). Para garantir que um objeto nunca seja colocado em um estado inválido, uma abordagem de Simple Factory pode ser combinada com a restrição do construtor como privado, forçando a criação através de um método que aplica validações e retorna um resultado encapsulado (Envelope Result) ([00:15:17](#)) ([00:18:13](#)). Wallace Silva e Farley Souza contribuíram com questões sobre a complexidade de manutenção e a generalização de classes quando se usam múltiplos construtores ([00:19:45](#)) ([00:23:57](#)).
- **Experiências com Simple Factory:** Gustavo Ferraz Fontes compartilhou que sua equipe utiliza o Simple Factory extensivamente, especialmente em objetos complexos, pois aumenta a legibilidade e expressividade, facilitando o entendimento de qual construtor usar. Eles usam o padrão para criar objetos *default* ou vazios, como "money zero," para comparações e verificações de nulidade, o que melhora a expressividade do código ([00:26:30](#)).

- **Factory Method (Método de Fábrica):** O Factory Method apresenta maior complexidade e mais abstrações do que o Simple Factory. A intenção continua sendo a criação de objetos, mas o Creator (a factory) retorna uma factory que é especialista em criar um produto concreto, levando a uma factory por produto concreto ([00:29:04](#)). Este padrão leva a sério o princípio de Responsabilidade Única (Single Responsibility Principle) e o princípio Aberto/Fechado (Open/Closed Principle), pois a adição de um novo tipo de produto geralmente requer apenas a criação de duas novas classes sem alterar o código cliente ([00:40:34](#)).
- **Tradeoffs e Complexidade do Factory Method:** Gustavo Ferraz Fontes exemplificou o uso do Factory Method em um ambiente bancário complexo, onde a necessidade de construir calculadoras complexas justificava o padrão "by the book" (segundo o livro) ([00:34:03](#)). Francisco Schneider concordou que o uso de padrões mais complexos é um *tradeoff* que se paga em cenários complexos ou em times maiores, onde evita-se o "mini caos" de conflitos de *merge* ao isolar as responsabilidades ([00:35:29](#)) ([00:37:47](#)).
- **Abstract Factory (Fábrica Abstrata):** O Abstract Factory é considerado uma expansão ou "V2" do Factory Method, pois resolve muito bem uma *família de produtos* ou um grupo de objetos correlacionados, em vez de apenas um único tipo de produto ([00:42:09](#)). O cliente interage apenas com as abstrações dos produtos ([00:43:34](#)). Uma consequência negativa do Abstract Factory é que o suporte a novos tipos de produtos exige a alteração da classe de fábrica concreta para atender ao novo objeto, o que pode violar o princípio Open/Closed (fechado para modificação), mas isso é um *downside* aceito devido aos benefícios ([00:46:49](#)) ([00:50:36](#)).
- **Padrão Builder (Construtor):** A intenção do Builder é o oposto do Factory, invertendo o jogo ao dar o poder de montagem do objeto ao código cliente ([00:53:37](#)). Embora o cliente tenha liberdade para montar o objeto, isso gera uma complexidade maior para garantir que o objeto nunca seja colocado em um estado inválido ([00:55:17](#)). O Builder positivo promove um controle fino sobre a criação e a responsabilidade única, mas pode aumentar a complexidade do código ([00:56:45](#)).
- **Uso e Definição do Padrão Builder:** O padrão Builder é amplamente utilizado em configurações de API, construção de *queries* com `FCEcore`, `StringBuilder` e logs, sendo um padrão importante de criação de objetos. Diferentemente do padrão Factory, o Builder concede ao cliente o poder e a responsabilidade sobre a montagem do objeto ([00:58:26](#)). Para alguns casos complexos, como a criação de objetos para boletos que podem ter variações e obrigatoriedades específicas, o Builder pode se tornar complexo, pois

transfere a responsabilidade da garantia de métodos essenciais e de um estado válido para o cliente ([01:10:28](#)).

- **Discussão sobre o Singleton e Concorrência:** O padrão Singleton foi introduzido como o próximo padrão criacional a ser analisado, e a primeira implementação apresentada levantou preocupações sobre compartilhamento de estado, alocação em memória e potencial não **thread-safe**. O compartilhamento de estado em um objeto global significa que todos compartilham a mesma instância, o que pode levar a um comportamento não **thread-safe** e, potencialmente, violar o princípio da responsabilidade única, pois a classe resolve o reuso e a alocação de memória ([01:13:11](#)). A intenção do Singleton é que exista apenas uma instância de determinada classe, acessível por vários clientes, e que sempre seja entregue a mesma instância ([01:14:36](#)).
- **Melhorias e Riscos na Implementação do Singleton:** Para resolver o problema de ser não **thread-safe** na implementação original do Singleton, a adição de um **lock** foi sugerida, transformando a classe em **thread-safe**, mas com uma penalidade de performance em cenários de concorrência ([01:16:11](#)). Uma solução mais performática é o uso de **lazy* *loading**, uma funcionalidade antiga no .NET (introduzida no .NET 4 em 2010), que é considerada **thread-safe** e oferece melhor desempenho do que o **lock**. Uma situação de risco foi mencionada onde o reuso implícito de um serviço **scoped** dentro do mesmo escopo de requisição resultou na recuperação de um valor de método antigo, o que impactou negativamente o serviço ([01:17:41](#)).
- **Abordagem Moderna do Singleton e o Princípio da Responsabilidade Única:** As abordagens manuais de Singleton com **lock** ou **lazy** violam o Princípio da Responsabilidade Única, pois a classe se torna responsável tanto por suas funcionalidades quanto pela gestão da entrega de suas instâncias. Atualmente, o método mais usual e moderno para implementar Singleton é através da injeção de dependência do **framework** (como o .NET), que garante o controle **thread-safe** e uma entrega mais performática de instâncias únicas, eliminando a necessidade de controle manual ([01:19:06](#)). Embora o **framework** garanta que todos recebam a mesma instância, é necessário cuidado, pois a classe Singleton ainda pode ter referências a outras classes, o que pode causar problemas de concorrência internos ([01:20:33](#)).
- **Transição para Padrões Estruturais e o Decorator:** O padrão Decorator foi introduzido como o primeiro padrão estrutural, focando em composição para adicionar novas funcionalidades ([01:21:59](#)). Padrões estruturais visam promover a inclusão de novas funcionalidades através de composição. O

exemplo central usado para discutir o Decorator foi a necessidade de adicionar cache a uma chamada de **endpoint** existente sem modificar o código do cliente ([01:23:39](#)).

- **Aplicação e Vantagens do Decorator:** A solução proposta para adicionar cache sem quebrar o código cliente foi encapsular a chamada em um novo serviço ou classe, buscando primeiro no cache ([01:25:13](#)). Essa abordagem é consistente com o Decorator, que permite adicionar novos comportamentos (como cache ou log/trace) através da composição de objetos (**wrapper**) ([01:26:52](#)). O Decorator é vantajoso por evitar que a classe original (por exemplo, o serviço de tempo) faça mais do que deveria (violando SOLID) e permite o rastreamento de ações (como ida ao banco) sem modificar o repositório ([01:29:24](#)).
- **Detalhes da Implementação do Decorator com a Lib de Injeção de Dependência:** O Decorator utiliza uma abstração (interface) implementada tanto pelo componente concreto (**core**) quanto pelo próprio **Decorator**, que por sua vez encapsula o componente interno ([01:29:24](#)). Para facilitar a injeção e a criação de **wrappers**, uma biblioteca externa que oferece o método `.Decorate`` é sugerida, simplificando o processo de garantir que a injeção entregue o **wrapper** em vez da implementação **core** ([01:30:57](#)). Esta lib facilita a criação de **pipelines** ou **steps** (como checagem de permissão) antes da execução do **command handler** concreto ([01:34:02](#)).
- **Ordem de Execução e Complexidade do Decorator:** A ordem em que múltiplos **Decorators** são aplicados é definida no momento da injeção de dependência, usando múltiplas linhas de `.Decorate`` para compor um **wrapper** dentro de outro **wrapper** ([01:38:56](#)) ([01:45:50](#)). No entanto, ter muitos **Decorators** sequenciais aumenta a complexidade e pode indicar que o Chain of Responsibility, um padrão comportamental que será discutido posteriormente, seria uma solução mais apropriada para a criação de uma cadeia de execução ([01:48:01](#)). O principal benefício do Decorator é que ele permite a extensibilidade do código, adicionando novas funcionalidades (como Circuit Breaker) sem a necessidade de modificar o código cliente, que continua solicitando a interface original ([01:41:45](#)) ([01:53:14](#)).
- **O Padrão Façade (Fachada):** O próximo padrão a ser discutido é o Façade, que é reconhecido como um padrão com potencial para auxiliar em necessidades específicas, embora seja frequentemente negligenciado. O Façade é um padrão que a audiência provavelmente já utiliza, mas sob um nome diferente ([01:56:22](#)).
- **Discussão sobre problemas em trechos de código e a falta de encapsulamento:** Francisco Schneider iniciou a discussão perguntando sobre

os problemas visíveis em um trecho de código, no qual Wallace Silva apontou haver "muita checagem" para uma única classe. Francisco Schneider confirmou que o problema central é a raiz de um problema maior no final e destacou a questão de o código não estar encapsulado em um método ou serviço, o que força a repetição do processo de verificação de elegibilidade para um empréstimo sempre que necessário. Eles observaram que isso é problemático, pois exige que os desenvolvedores se lembrem de quais classes instanciar, quais métodos chamar e qual é a ordem correta das operações ([01:57:45](#)).

- **Aplicação do Conceito Fail Fast e Introdução ao Padrão Facade:** Francisco Schneider abordou o conceito de "Fail Fast" (falhar o mais rápido possível), mencionando que a aplicação dessa premissa economiza recursos e é comprometida quando o código não está encapsulado. A intenção do padrão Facade (Fachada) é prover uma interface única para um subsistema, agindo como um orquestrador de múltiplos subsistemas, contextos e serviços. O diagrama oficial do padrão Facade ilustra que o cliente interage exclusivamente com a fachada, e é esta quem se conecta aos serviços necessários, que podem ser entendidos como módulos ou subsistemas ([01:59:26](#)).
- **Intenção do Design do Padrão Facade e Implementação Prática:** O diagrama do padrão Facade apresenta caixas em branco para os subsistemas que a fachada acessa, o que é proposital, pois o cliente não se importa com os sistemas dos quais a fachada depende. O cliente simplesmente aciona a fachada e pede algo; ela, por sua vez, realiza o intermédio para que o trabalho seja concluído. Francisco Schneider mostrou uma refatoração na qual um serviço de fachada foi criado, com um método `isElegible` que encapsula o comportamento que antes estava disperso, e sugeriu que nomes como `*financing handler*` ou `*financing application service*` seriam mais familiares ([02:00:54](#)).
- **Uso e Nomenclatura do Padrão Facade no Cotidiano dos Desenvolvedores:** Francisco Schneider expressou a convicção de que o padrão Facade é amplamente utilizado pelos desenvolvedores, embora possa ter recebido novas nomenclaturas. Gustavo Ferraz Fontes confirmou que a equipe dele utiliza a nomenclatura "fachada" e a empregou em um legado para simplificar modificações e encapsular código durante um processo de refatoração ([02:02:29](#)). Eles usaram o padrão, inclusive, em um processo de estrangulamento (do inglês, `*strangling*`) a nível de design para tornar fluxos complexos mais testáveis ([02:04:25](#)).
- **Estratégia de Refatoração de Legado e Domínios Ricos usando Facade:** Francisco Schneider explicou que a introdução do padrão Facade em um

legado com um domínio pobre e deficiente é uma alternativa eficiente para modernizar o sistema sem a necessidade de criar novas APIs ou descartar todo o código ([02:05:46](#)). Um domínio rico é caracterizado por ter mais testes de unidade do que testes de integração, o que o torna mais testável ([02:07:07](#)). Eles descreveram a abordagem de estrangulamento: a fachada foi introduzida gradativamente, chamando inicialmente a classe legada, até que novas classes modernas nasceram e o componente passou a redirecionar as operações para o novo código, permitindo a eventual remoção do legado ([02:08:29](#)).

- **Implementação e Controle do Estrangulamento de Legado:** Natan Barros questionou se o controle de quanto o tráfego iria para o código legado versus o moderno era feito por versionamento de API, mas Francisco Schneider esclareceu que o método adotado foi a migração gradativa de operações dentro da mesma versão ([02:11:34](#)). Eles migraram as operações aos poucos, de modo que todos os clientes consumiam as mesmas operações, mas a execução era progressivamente transferida do código antigo para o novo ([02:12:56](#)). Esse processo é refatoração em sua essência, pois melhora o design e a orientação a objetos sem alterar o comportamento final para o cliente ([02:14:07](#)).
- **Vantagens de Negócio e Métricas para Vender a Refatoração:** O ponto mais crítico para implementar a refatoração é conseguir "vender" a necessidade dela para o negócio, muitas vezes mostrando ganhos não técnicos ([02:14:07](#)). No caso discutido, a refatoração foi vendida porque a introdução de novas funcionalidades estava demorando muito e havia uma alta taxa de bugs, problemas que doíam para os gerentes de projeto e superiores ([02:16:34](#)). Natan Barros sugeriu que métricas como a quantidade de commits e autores por arquivo, obtidas via ferramentas como o Git, podem ser usadas para colher evidências e demonstrar o valor da modernização ([02:18:04](#)).
- **Consequências do Uso do Padrão Facade e Implementação de Controles Físicos:** Os benefícios do Facade incluem facilitar o código cliente através da abstração e atender ao princípio aberto/fechado, pois o cliente se relaciona apenas com a fachada ([02:18:04](#)). No entanto, uma consequência negativa é que o cliente, por padrão, poderia ignorar a fachada e acessar a classe legada diretamente, a menos que existam controles físicos, como modificadores de acesso, para impedir isso. Gustavo Ferraz Fontes explicou que em seu caso, o bloqueio direto não era viável inicialmente, mas eles garantiram que certas classes não tivessem referência a outras *via* testes de unidade no tempo de *build* ([02:19:42](#)).

- **O Padrão Facade no Framework .NET e Encerramento:** Francisco Schneider mencionou que o padrão Facade também está presente no .NET, sendo exemplificado pelo `DatabaseFacade` do Entity Framework (EF), que abstrai a complexidade de gerenciar conexões, transações e comandos SQL brutos. Ele encerrou a discussão agradecendo a participação e mencionando que o grupo se reunirá para finalizar o assunto na quarta-feira ([02:21:08](#)).

Revise as anotações do Gemini para checar se estão corretas. [Confira dicas e saiba como o Gemini faz anotações](#)

*Como está a qualidade de **destas observações**? [Responda a uma breve pesquisa](#) para nos dar seu feedback, incluindo o quanto as observações foram úteis para o que você precisa.*